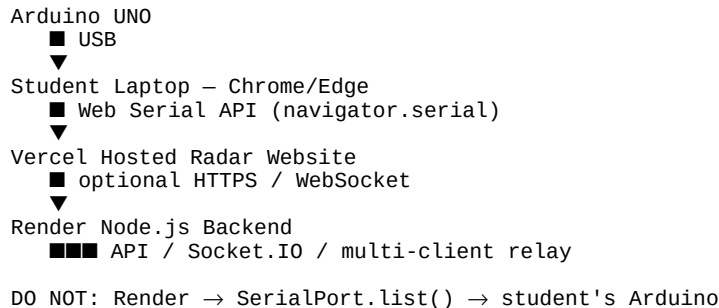


Arduino Radar Web Application

Corrected Deployment Guide — Vercel + Render + Web Serial

Important: Render cannot access the student's Arduino USB port. The browser on the student's laptop must handle Arduino Serial using the Web Serial API. Render should handle only cloud/API/WebSocket work.

1. Correct Architecture



2. Why Your Render Error Happened

Your Render log shows **spawn udevadm ENOENT** while listing serial ports. That means the deployed server is trying to find Linux USB/serial devices. Your Arduino is connected to your laptop, not to Render. Therefore the cloud server must not run `SerialManager/SerialPort.list()` for the student's Arduino.

3. Render Setup

Environment: **Node**

Root Directory: **server** (if `server/package.json` exists there)

Build Command: **npm install**

Start Command: **node server.js**

Use **`process.env.PORT`**; do not create a physical COM-port setting on Render.

Use cloud-only server code such as:

```
const express = require("express");
const http = require("http");
const cors = require("cors");
const { Server } = require("socket.io");

const app = express();
const server = http.createServer(app);
const FRONTEND_ORIGIN = "https://YOUR-RADAR.vercel.app";

app.use(cors({ origin: FRONTEND_ORIGIN, methods: ["GET", "POST"] }));

const io = new Server(server, {
  cors: { origin: FRONTEND_ORIGIN, methods: ["GET", "POST"] }
});

io.on("connection", socket => {
  console.log("Client connected:", socket.id);
  socket.on("radar-data", data => socket.broadcast.emit("update-radar", data));
});

app.get("/", (req, res) => res.json({ok:true, service:"Arduino Radar Backend"}));

const PORT = process.env.PORT || 5000;
server.listen(PORT, () => console.log("Radar backend listening on". PORT));
```

Remove from deployed Render code: `SerialPort.list()`, automatic COM scanning, `udevadm`-dependent serial detection, and any attempt to open the student's USB port.

4. Vercel Frontend — Direct Arduino Connection

The hosted website reads the Arduino directly in the student's browser at 9600 baud:

```
let port = null;
let reader = null;
```

```

let buffer = "";

async function connectArduino() {
  if (!("serial" in navigator)) {
    alert("Use Google Chrome or Microsoft Edge.");
    return false;
  }

  try {
    const ports = await navigator.serial.getPorts();

    // Previously authorized port
    port = ports.length ? ports[0] : await navigator.serial.requestPort();

    await port.open({ baudRate: 9600 });

    const decoder = new TextDecoderStream();
    port.readable.pipeTo(decoder.writable);
    reader = decoder.readable.getReader();

    readSerialStream();
    return true;
  } catch (err) {
    console.error("Arduino connection failed:", err);
    return false;
  }
}

async function readSerialStream() {
  while (reader) {
    const { value, done } = await reader.read();
    if (done) break;

    if (value) {
      buffer += value;
      const lines = buffer.split("\n");
      buffer = lines.pop();

      for (const line of lines) {
        processArduinoLine(line.trim());
      }
    }
  }
}

function processArduinoLine(line) {
  const p = line.split(",");
  if (p.length !== 2) return;

  const angle = Number(p[0]);
  const distance = Number(p[1]);
  if (!Number.isFinite(angle) || !Number.isFinite(distance)) return;

  drawRadarPoint(angle, distance);

  // Optional cloud relay
  if (socket && socket.connected)
    socket.emit("radar-data", { angle, distance });
}

```

5. F4 Fallback

A first-time browser cannot silently choose an arbitrary USB port. After permission has been granted, `getPorts()` may find the authorized Arduino automatically. If it does not, F4 should open the browser's port selector:

```

window.addEventListener("keydown", event => {
  if (event.key === "F4") {
    event.preventDefault();
    connectArduino();
  }
});

```

6. Student Workflow

1. Upload the Arduino C++ code to UNO.
2. Check Serial Monitor if required; data should look like 90,45.3.
3. Close Serial Monitor / Serial Plotter.
4. Open the HTTPS Vercel website in Chrome or Edge.

5. Website attempts automatic connection to an authorized Arduino.
6. If no authorized port is found, press F4 and select Arduino.
7. Radar reads the 9600-baud data directly and starts drawing.

7. Vercel Deployment

Import the GitHub repository into Vercel. Deploy the frontend over HTTPS. Use the Vercel production URL as the frontend origin in Render. The frontend must not use `http://localhost:5000` in production.

8. Environment Variables

Variable	Example	Where
FRONTEND_ORIGIN	<code>https://your-radar.vercel.app</code>	Render
BACKEND_URL	<code>https://your-radar.onrender.com</code>	Vercel/frontend
BAUD_RATE	9600	Frontend/config; not a Render COM port

9. Troubleshooting

Problem	Cause	Fix
udevadm ENOENT on Render	Cloud is trying to list local serial ports.	Remove server-side serial scanning; use Web Serial in browser.
navigator.serial undefined	Unsupported browser or not HTTPS.	Use Chrome/Edge and the HTTPS Vercel URL.
Port busy	Serial Monitor/Plotter is open.	Close it, then reconnect.
No port appears	No permission or Arduino disconnected.	Press F4 and select Arduino; check USB cable/driver.
Hosted site fails but local works	Production code still uses localhost or server-side serial.	Use navigator.serial for USB and production Render URL only for cloud networking.
Render sleeps	Free cloud instance may sleep.	This affects optional cloud networking, not direct browser-to-Arduino USB.

10. Final Checklist

- Arduino outputs angle,distance at 9600 baud.
- Serial Monitor is closed before connecting from the website.
- Vercel website is HTTPS.
- Chrome/Edge supports Web Serial.
- Browser uses navigator.serial for Arduino USB.
- F4 opens the browser port selector as fallback.
- Render does NOT run `SerialPort.list()` or server-side USB detection.
- Render CORS allows the exact Vercel origin.
- Frontend uses the production Render URL, not localhost.
- Radar draws directly from browser serial data.

Key point: Vercel hosts the website, Render hosts the optional Node.js cloud backend, and the student's browser is the USB bridge to the Arduino.